Tema 04: Gestión de E/S

METODOLOGÍA DE LA PROGRAMACIÓN

GRADO EN INGENIERÍA INFORMÁTICA

DECSAI

UNIVERSITAS GRANATENSIS · 1531 · PLUS ULTRA

Índice

- Argumentos de main.
- Flujo de E/S.
- Ficheros.
- Ficheros de texto y binarios.

Objetivos

- Conocer el significado de las variables argc y argv que tiene la función main.
- Entender el concepto de flujo de E/S y buffer de entrada y salida.
- Conocer las diferentes maneras existentes de pasar información al programa distintas al uso de cin.
- Entender cómo el lenguaje de programación se comunica con los ficheros almacenados en el disco duro y los mecanismos estándares existentes para la lectura y escritura de datos en ellos.
- Estudio de las diferentes representaciones de la información que pueden contener los ficheros.

Argumentos de main

- Formato de la función main:

```
#include <iostream>

using namespace std;

int main(int argc, char ** argv) {
 /*codigo*/

    return 0;
}
```

- **argv**: argumentos de la llamada (cada posición es un char *).
- **argc**: número de argumentos.

Argumentos de main

EJEMPLO:

- **Ejecutamos:** <ejecutable> -a –v –f
- **Contenido de las variables:**

argc: 4

argv:

Ruta/<ejecutable>	-a	-v	-f

- El primer argumento es la ruta del ejecutable.

Argumentos de main

DECSAI

EJECUTAMOS EL SIGUIENTE CÓDIGO:

```
#include <iostream>

using namespace std;

int main(int argc, char ** argv) {

    cout << "Hay: " << argc << " argumentos" <<  endl;
    cout << "Los argumentos son: ";
    for(int i = 0; i < argc; i++){
        cout << string(argv[i]) << ", ";
    }

    return 0;
}
```

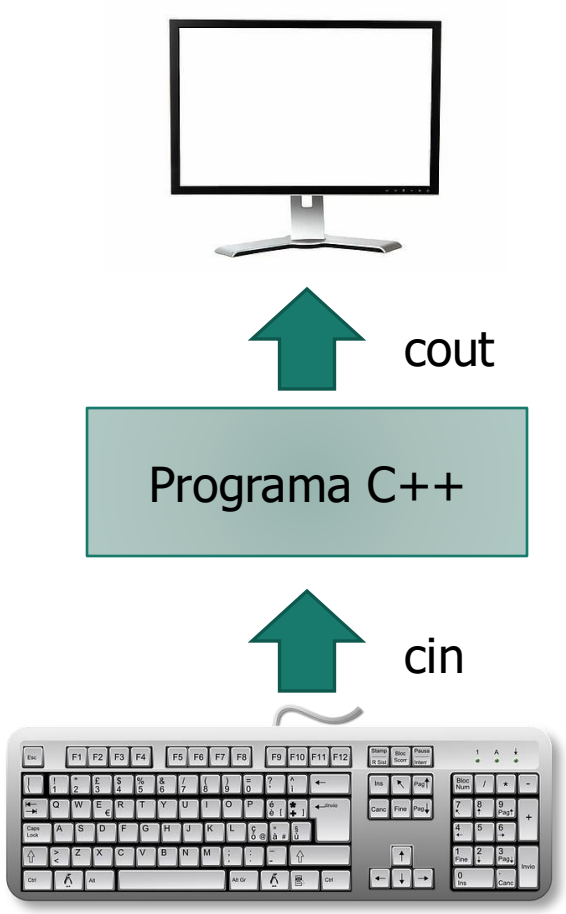
Resultado:

```
Hay: 4 argumentos
Los argumentos son: E:\pruebacodeblock\pruebaficheros\bin\Debug\pruebaficheros.exe, -a, -v, -f,
Process returned 0 (0x0)   execution time : 1.578 s
Press any key to continue.
```

Flujos de E/S

- Los flujos conectan nuestros programas con los dispositivos de entrada/salida (E/S) del ordenador.
- Son secuencias de bytes que se mandan entre los dispositivos E/S y nuestro programa.
- Permiten al programa recibir datos externos y también mostrar resultados al usuario.
- Ya conocéis algunos: cout y cin (pantalla y teclado).
- Representan una conexión entre el programa y su exterior

Flujos de E/S



Flujos de E/S

- Se envían y reciben bytes.
- C++ proporciona herramientas de bajo y alto nivel:
  - **Bajo nivel**: Se especifica el número de bytes a transmitir.
  - **Alto nivel**: Se especifica el tamaño a transferir usando los tipos de datos conocidos: enteros, float, caracteres, cadenas, etc.

Flujos de E/S

- En la biblioteca <iostream>
  - **cin**: flujo de entrada.
  - **cout**: flujo de salida.
  - **cerr**: flujo de error sin búfer.
  - **clog**: flujo de error estándar con búfer.
- En la biblioteca <fstream>
  - Funciones para el manejo de ficheros.

Flujo de E/S. Redirección del flujo a fichero

- Se hace al ejecutar el fichero.

- Redirigir el flujo de entrada cin:
  - <ejecutable> < datosentrada.txt

- Redirigir el flujo de salida cout:
  - <ejecutable> > datossalida.txt

- Redirigir el flujo de error cerr:
  - <ejecutable> &> errores.txt

- Redirigir el flujo de salida al flujo de entrada de otro programa:
  - <ejecutable1> | <ejecutable2>

Flujos de E/S

- Algunas funciones para formatear la salida:

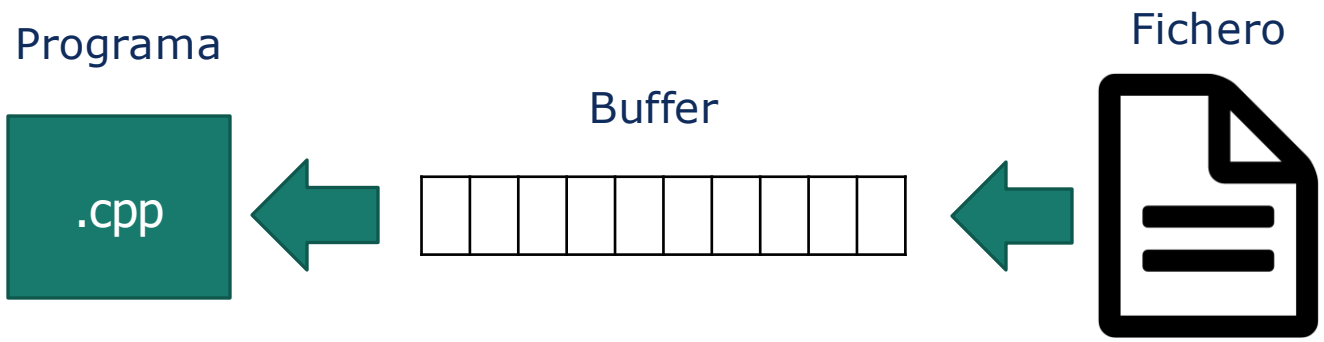
```
#include <iostream>
#include <iomanip>

using namespace std;

int main(){
    cout << scientific << 3232.345345424 << endl;
    cout << setw(10) << 23424 << endl;
    cout << fixed << setprecision(3) << 3.45354353 << endl;
}
```

E:\pruebacodeblock\pruebaficheroscodigos\formateocout.exe

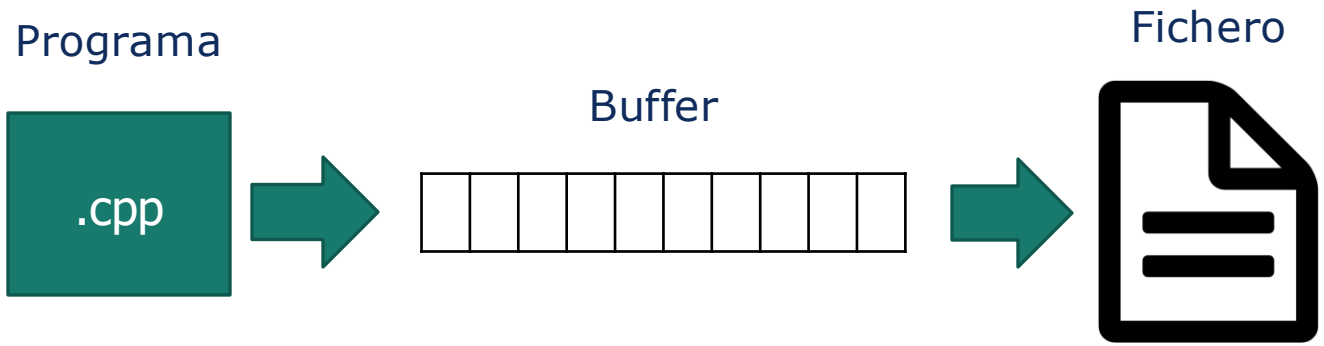
```
3.232345e+003
     23424
3.454

Process returned 0 (0x0)   execution time : 0.048 s
Press any key to continue.
```

Flujos de E/S. Buffer de E/S

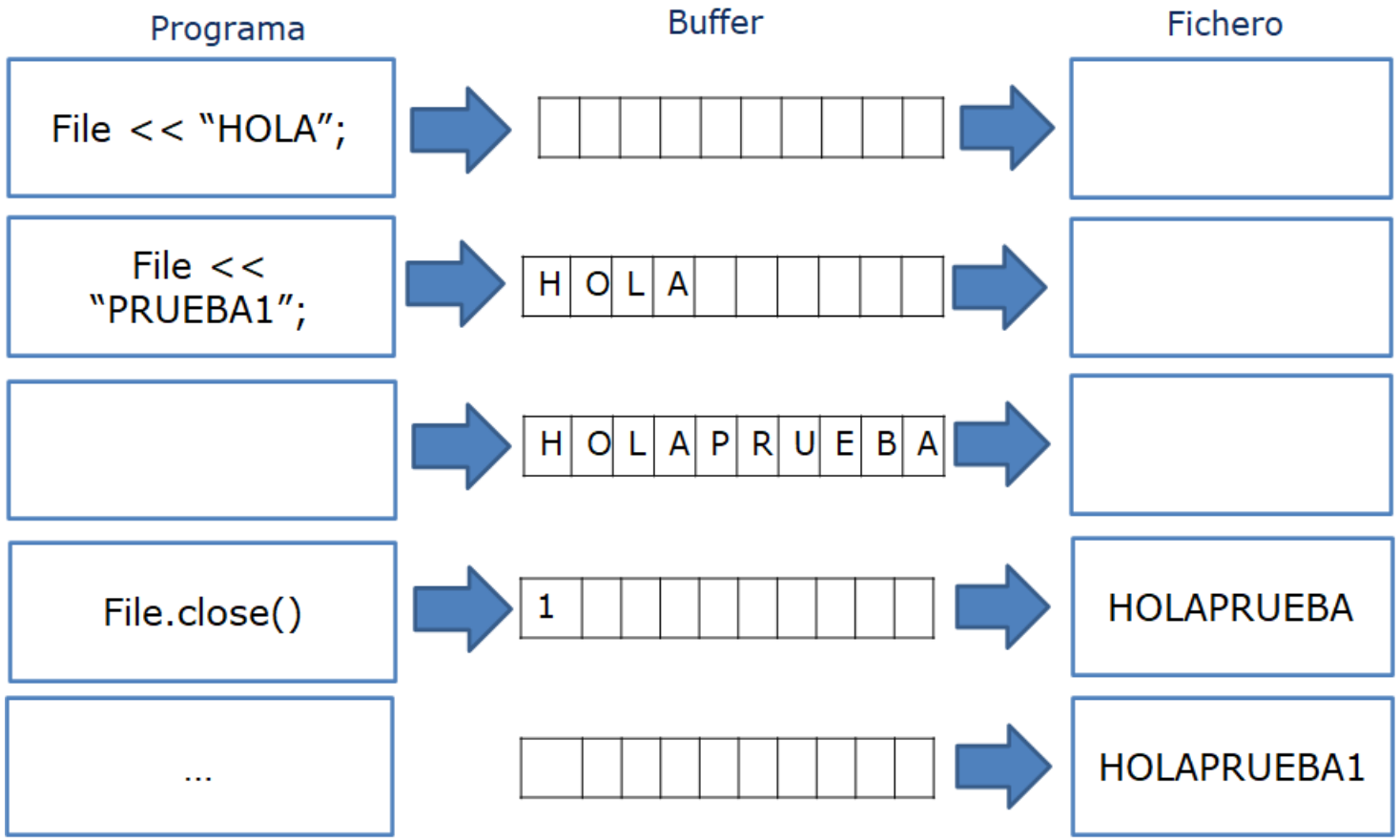
Cuando leemos datos:



Cuando escribimos datos:



Flujos de E/S. Ejemplo de escritura



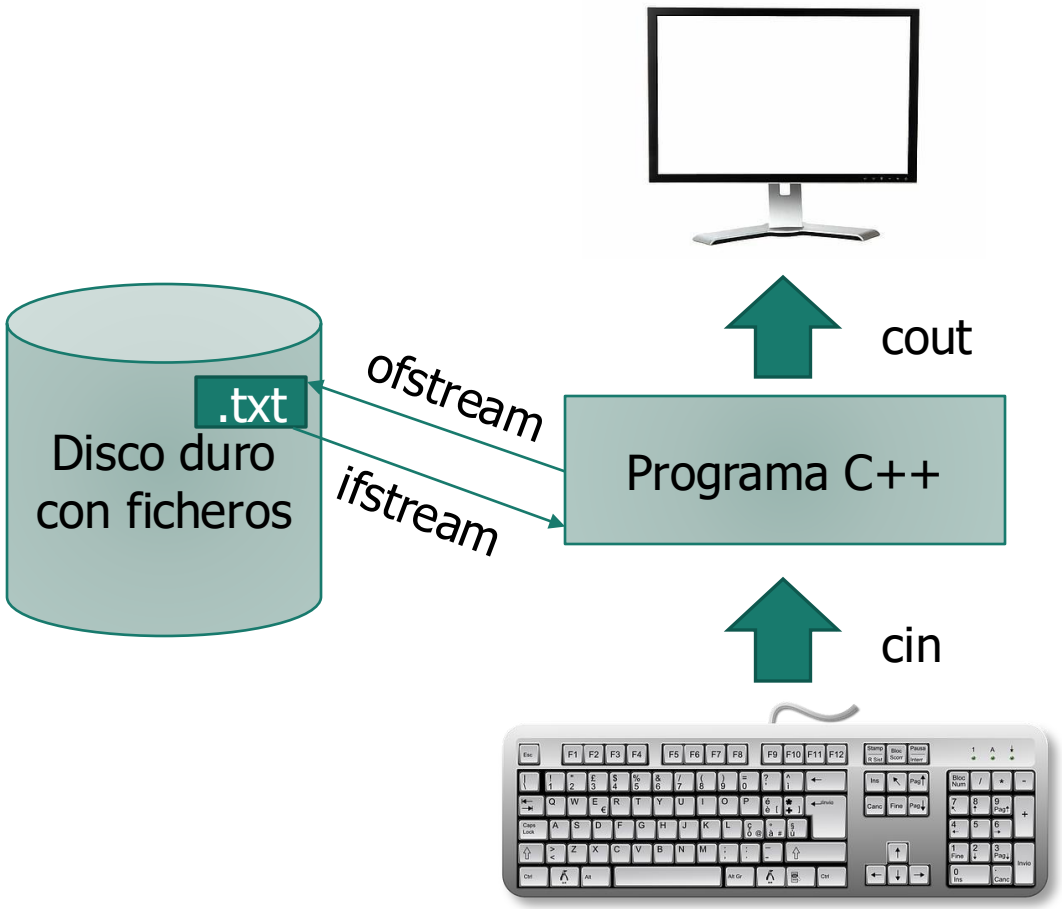
Ficheros

- Utilizamos la biblioteca <fstream>.
- Pasos lectura:
  - Declaramos un objeto de la clase ifstream.
  - Abrimos el fichero.
  - Leemos los datos utilizando los diferentes operadores y funciones disponibles.
  - Cerramos el fichero.

Ficheros

- Utilizamos la biblioteca <fstream>.
- Pasos escritura:
  - Declaramos un objeto de la clase ofstream.
  - Abrimos el fichero.
  - Escribimos los datos utilizando los diferentes operadores y funciones disponibles.
  - Cerramos el fichero.

Ficheros


.txt
Disco duro
con ficheros
ofstream
ifstream
Programa C++
cout
cin

Ficheros

- El acceso a los datos del fichero es secuencial. Una vez se lee un dato el apuntador pasa al siguiente y no se puede volver de forma directa hacia atrás.
- Intentaremos hacer lecturas eficientes: Hacer las menos pasadas al fichero posibles.
- Todos los datos almacenados en un fichero son tratados como caracteres cuando se leen.

Ficheros

- No es recomendable por temas de eficiencia y legibilidad de código pero se puede usar el método seekg para mover el apuntador del fichero.

- Nos colocamos en el n-esimo byte del fichero:

```
fi.seekg(n);
```

- Avanzamos n bytes el apuntador del fichero:

```
fi.seekg(n,ios::cur);
```

- Colocamos el apuntador al final del fichero:

```
fi.seekg(0,ios::end);
```

Ficheros

- ¿Y si sólo queremos saber la posición?
- Métodos tellg y tellp.
- **En ifstream**:
  - Tellg y seekg (g viene de get).
- **En ofstream**:
  - Tellp y seekp (p viene de put).

Ficheros. Operadores << y >>

- Se utilizan sobre el objeto ifstream/ofstream creado para leer/escribir datos.
- **<<:** Operador de escritura.
  - <objeto ofstream> << <datos a almacenar>
- **>>:** Operador de lectura.
  - <objeto ifstream> >> <variable donde almacenar la información>
- El dato leído debe coincidir con el tipo de la variable.

Ejemplo de lectura/escritura

```
#include <fstream>
#include <iostream>

using namespace std;

int main(){
    ifstream fi;
    ofstream fo;
    string cadena;
    float numero;
    int entero;

    fi.open("entrada.txt");
    fi >> cadena >> numero >> entero;
    fi.close();

    cout << cadena << " " << numero << " " << entero << endl;

    fo.open("salida.txt");
    fo << cadena << " " << numero << " " << entero << endl;
    fo.close();
}
```

PROBLEMA: ¿Y si el fichero no existe o se produce un error?

entrada: Bloc de notas

Archivo Edición Formato Ver Ayuda

```
hola 5.6 7
```

salida: Bloc de notas

Archivo Edición Formato Ver Ayuda

```
hola 5.6 7
```

```
hola 5.6 7

Process returned 0 (0x0)   execution time : 0.049 s
Press any key to continue.
```

Ejemplo de lectura/escritura (II)

```
string a;
int x;
char y;

int main ()
{
ifstream mi_fichero_entrada; //  flujo de entrada
ofstream mi_fichero_salida; //   flujo de salida

mi_fichero_salida.open("fSalida.txt");
mi_fichero_entrada.open("datosEntrada.txt");

...
if (mi_fichero_entrada.is_open()){
   mi_fichero_entrada >> a; // lee hasta encontrar separador
   mi_fichero_entrada >> x >> y ; // lee entero  char
}

if (mi_fichero_salida.is_open()){
   mi_fichero_salida << "El␣numero␣de␣lineas␣es" << n_lineas <<
   endl;
   for (int i=0; i<n_lineas;i++)
      mi_fichero_salida << "linea␣"<< i << endl;
mi_fichero_entrada.close();
mi_fichero_salida.close();
```

Comprobación de final de fichero

- Nos va a interesar crear bucles que lean un fichero entero y vayan procesando datos. Podemos procesar:
  - Dato a dato.
  - Línea a línea: En este caso es interesante que cada línea sea una entrada.

- Cuando se llega al final del fichero: Se enciende la bandera de estado EOF.

- Podemos usar el método .eof() para comprobar si un determinado objeto ifstream o ofstream tiene encendida la bandera de estado EOF.

Comprobación de errores y banderas de estado

- **.is_open():** ¿Se ha abierto correctamente el fichero?
- **.clear():** Elimina los flags de error.
- **.good():** ¿Hay flags de error activos?
- **.bad():** ¿Hay algún problema en la conexion con el fichero?
- **.fail():** ¿Hay algún problema con cómo se están leyendo los datos?
- **.eof():** ¿Se ha intentado leer después de llegar al final del fichero?

Ejemplo de lectura/escritura (III)

```
#include <fstream>
#include <iostream>
using namespace std;

int main(){
    /* Leer las 50 primeras palabras de un fichero y/o parar cuando acabe el fichero*/
    ifstream fi;
    string * cadenas;
    bool findefichero = false;
    cadenas = new string [50];
    int numpalabras=0;

    fi.open("textolargo.txt");
    if(fi.is_open()){
        while(!fi.eof() && numpalabras < 50){
            fi >> cadenas[numpalabras];
            numpalabras++;
        }
    }
    fi.close();
    for(int i = 0; i < numpalabras; i++){
        cout << cadenas[i] << endl;
    }
}
```

Lectura de un solo carácter/línea

- Leer un solo carácter:
  - Leer entero con posición en la tabla ASCII: int get ();
  - Leer un carácter: istream & get (char & c);

- Leer una línea:
  - istream & getline (char * s, int n);
  - istream & getline (char * s, int n, char delim);
  - istream & getline (istream & fi, std::string & s);

Ejemplo: Leer líneas de fichero

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main(){
    /* Leer las 50 primeras líneas de un fichero y/o parar cuando acabe el fichero*/
    ifstream fi;
    string * cadenas;
    bool findefichero = false;
    cadenas = new string [50];
    int numlineas=0;

    fi.open("textolargo.txt");
    if(fi.is_open()){
        while(getline(fi,cadenas[numlineas])
              && numlineas < 50){
            numlineas++;
        }
    }
    fi.close();
    cout << "Numero de lineas:" << numlineas<< endl;
    for(int i = 0; i < numlineas; i++){
        cout << cadenas[i] << endl;
    }
}
```

E:\pruebacodeblock\pruebaficheroscodigos\lecturayescrituralineas.exe

```
Numero de lineas:14
En lo que concierne al sistema operativo un archivo es, en la mayor├¡a de los casos,
simplemente un flujo unidimensional de bytes, que es tratado por el sistema operativo
como una ├╢nica unidad l├│gica. Un archivo de datos inform├ítico normalmente tiene un tama├▒o,
que generalmente se expresa en bytes; en todos los sistemas operativos modernos, el tama├▒o puede
ser cualquier n├╢mero entero no negativo de bytes hasta un m├íximo dependiente del sistema. Depende
del software que se ejecuta en la computadora el interpretar esta estructura b├ísica como por ejemplo
 un programa, un texto o una imagen, bas├índose en su nombre y contenido. Los tipos especiales de
archivos, como los nodos de dispositivo que representan simb├│licamente partes del hardware, no
consisten en un flujo de bytes y no tienen tama├▒o de archivo.
Los datos de un archivo inform├ítico normalmente consisten en paquetes m├ís peque├▒os
de datos (a menudo llamados registros o l├¡neas) que son individualmente diferentes
pero que comparten alg├╢n rasgo en com├╢n. Por ejemplo, un archivo de n├│minas puede
contener datos sobre todos los empleados de una empresa y los detalles de su n├│mina;
cada registro del archivo de n├│minas se refiere ├╢nicamente a un empleado, y todos loa

Process returned 0 (0x0)   execution time : 1.504 s
Press any key to continue.
```

De string a otros tipos de datos

- Para convertir un string leído en datos de otros tipos se pueden usar las siguientes funciones:
  - String a entero:

```
string a = "12";
int b = atoi(a.c_str());
```

  - String a float:

```
string a = "12.5";
float b = atof(a.c_str());
```

- .c_str() convierte un string en un char *.

Lectura/Escritura de matriz

Código de declarar matriz:

```
int ** declararMatriz(int filas, int columnas){
    int **matriz = new int*[filas];
    for (int i = 0; i < filas; i++) {
        matriz[i] = new int[columnas];
    }
    return matriz;
}
```

Lectura/Escritura de matriz

Código de lectura:

```
ifstream fi;
int ** matrizenteros;
string temp;
int filas;
int columnas;
fi.open("matriz.txt");
if(fi.is_open()){
    fi >> temp;
    fi >> filas;
    fi >> columnas;
    matrizenteros = declararMatriz(filas,columnas);
    for(int i = 0; i < filas; i++){
        for(int j = 0; j < columnas; j++){
            fi >> matrizenteros[i][j];
        }
    }
}
fi.close();
```

matriz: Bloc de notas

Archivo Edición Formato

```
Dimensiones: 3 4
4 5 6 3
4 5 6 4
3 4 5 3
```

Lectura/Escritura de matriz

Código de escritura:

```
ofstream fo;
fo.open("salidamatriz.txt");
if(fo.is_open()){
fo << "Dimensiones: " << filas << " " << columnas << endl;
for(int i = 0; i < filas; i++){
    for(int j = 0; j < columnas; j++){
        fo << matrizenteros[i][j] << " ";
    }
    fo << endl;
}
}
fo.close();
```

Resultado:

salidamatriz: Bloc de notas

Archivo Edición Formato Ver Ayuda

```
Dimensiones: 3 4
4 5 6 3
4 5 6 4
3 4 5 3
```

Lectura/Escritura de matriz

DECSAI

- IMPORTANTE:
  - Mantener una estructura única para cada tipo de dato almacenado.
    - Un mismo programa debe ser capaz de leer cualquier archivo que haya sido escrito por el mismo.
    - Establecer un estándar: única manera de representar los datos que utilizaremos para ese tipo de ficheros concreto.
  - Comprobar siempre que los ficheros se han abierto correctamente.
  - Introducir comprobadores de error.
    - **EJEMPLO**: En el programa de leer/escribir matriz sería deseable que se comprobara .eof en caso de que el fichero acabara abruptamente.

E/S sin formato

- **Método write**: introduce los bytes indicados en el buffer de salida.

```
char buffer[] = "hola a todos";
//Muestra los primeros 4 bytes de la cadena
cout.write(buffer,4);
```

- **Método read**: lee los bytes indicados del buffer de entrada.

```
cin.read(buffer,5);
//Lee 5 caracteres desde el teclado y los mete en buffer
cout << string(buffer);
```

- **Método gcount**: Devuelve el último número de datos leídos.

Lectura/Escritura binaria

- Utilizamos las funciones write y read sobre objetos ifstream y ofstream.
- Aunque el almacenamiento sea binario, hay que guardar una estructura.
- Podemos reconocer el tamaño de un tipo de dato mediante el uso del método sizeof().
- Podemos utilizar reinterpret_cast para transformar el tipo de dato en un conjunto de bytes a almacenar.

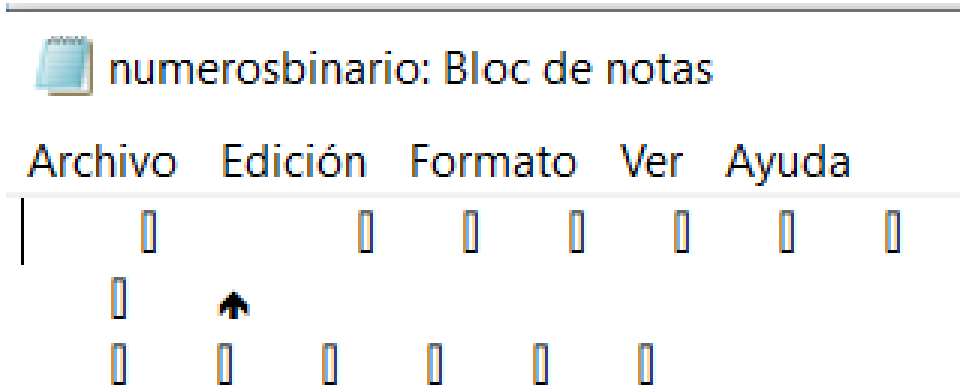
Lectura/Escritura binaria

- Escritura de fichero binario:

```
int * numeros;
const int TAM = 20;
numeros = new int [TAM];
for(int i = 0; i < TAM; i++){
    numeros[i] = i;
}

ofstream fo;
fo.open("numerosbinario.txt");
for(int i = 0; i < TAM; i++){
    fo.write(reinterpret_cast<const char *>(&numeros[i]), sizeof(numeros[i]));
}
fo.close();
```

Lectura/Escritura binaria

- Si ahora abrimos el fichero …



Lectura/Escritura binaria

- Lectura del quinto valor del fichero binario:

```
//Extraigo el quinto valor
ifstream fi;
fi.open("numerosbinario.txt");
fi.seekg(5*sizeof(int));
int quintovalor;
fi.read(reinterpret_cast<char *>(&quintovalor),sizeof(int));
cout << (quintovalor) << endl;
fi.close();
```

Lectura/Escritura binaria

- Lectura de todos los valores del fichero binario de manera secuencial:

```
//Leemos los valores uno a uno
ifstream fi2;
int valorleido;
fi2.open("numerosbinario.txt");
for(int i = 0; i < TAM; i++){
     fi2.read(reinterpret_cast<char *>(&valorleido),sizeof(int));
    cout << valorleido << endl;
}
```

Lectura/Escritura binaria

- Si añadimos tellg podemos ir viendo las posiciones en donde se encuentran los diferentes números:

```
//Leemos los valores uno a uno
ifstream fi2;
int valorleido;
fi2.open("numerosbinario.txt");
for(int i = 0; i < TAM; i++){
    cout << fi2.tellg() << " ";
     fi2.read(reinterpret_cast<char *>(&valorleido),sizeof(int));
    cout << valorleido << endl;
}
```

Lectura/Escritura binaria

Resultado de la ejecución:

```
0 0
5 1
9 2
13 3
17 4
21 5
25 6
29 7
33 8
37 9
41 10
45 11
49 12
53 13
57 14
61 15
65 16
69 17
73 18
77 19

Process returned 0 (0x0)   execution time : 1.447 s
Press any key to continue.
```

Referencias

- H. Deitel, P. Deitel. Como programar en C++. Pearson. Prentice Hall 2003.
- Cplusplus.com. https://www.cplusplus.com/